

Aşama 2: Saha Raporu Destekli LLM Agent

İlk gün geliştirdiğiniz tespit modelini kullanan bir LLM agent geliştirin.

Agent, verilen drone görüntüsünü araçların hareket verileri ve saha raporlarıyla birlikte değerlendirmeli. Hangi durumların dikkat gerektirdiğini, nedenleri ve dayandığı verilerle açıklamalı.

Senaryo

Bir üssü korumakla görevlisiniz. Çevredeki bölgeler gün boyu drone'larla izleniyor; sahadaki birimlerden de gözlem raporları geliyor. Elinizde bu izlemenin bir günlük çıktısı var: 40 drone görüntüsü, araçların hareket kayıtları ve saha raporları.

Verilen dosyalar

Dosya	İçerik
<code>images/</code>	40 drone görüntüsü.
<code>image_meta.json</code>	Her görüntünün piksel boyutları, çekim saati ve dört köşesinin koordinatları (WGS84).
<code>zones.json</code>	Merkez üssün ve 8 bölgenin adı ile koordinatları. Raporlardaki bölge adlarını bu dosyadan bulabilirsiniz.
<code>tracks.csv</code>	Araçların hareket kayıtları. Her <code>track_id</code> bir aracın son 2 saatteki konumlarını gösterir; kayıtlar 5 dakika aralıktır, <code>time</code> alanı gerçek saattir. Alanlar: <code>track_id</code> , <code>time</code> , <code>lat</code> , <code>lon</code> .
<code>field_reports.json</code>	Serbest metin saha raporları. Alanlar: <code>time</code> , <code>source</code> , <code>text</code> . Kaynak (<code>source</code>): <code>official</code> veya <code>third_party</code> . Konum, metinde koordinat veya bölge adı olarak geçer.

Uçtan uca örnek

Aşağıdaki değerler temsilidir; yöntemi göstermek içindir.

- Görüntüyü açın.** `img_000123` — `image_meta.json` 'a göre 1360×765 piksel, çekim saati **13:25**. Köşe koordinatları: sol üst (39.94510, 32.86200), sağ üst (39.94510, 32.86519), sol alt (39.94373, 32.86200).
- Aracı tespit edin.** 1. gün modeliniz bir kamyon buldu: kutu (610, 380, 60, 28) → merkez piksel (**640, 394**).
- Pikseli koordinata çevirin.**
 - boylam = **32.86350**
 - enlem = **39.94439**
- Hareket kaydını bulun.** `tracks.csv` 'de `time == 13:25` olan satırları süzün. (39.94441, 32.86353) konumundaki `T0187` yaklaşık 2,5 m mesafede — eşleşme bu.
- Hareketi çıkarın.** `T0187` 'nin 11:25–13:25 satırlarından: son yarım saatte hızı ~9 m/s; üsse uzaklığı 12:25'te 5,8 km iken 13:25'te 3,0 km — **üsse yaklaşıyor.**
- Raporlarla karşılaştırın.** 12:40 tarihli bir `third_party` rapor "39.944N 32.863E civarında bir kamyon" diyor — tespitinizle uyumlu, rapor bu aracı anlatıyor olabilir. Raporun iddiasını (tip, hareket, sayı) kendi bulgularınızla karşılaştırın; çelişki varsa raporu değil tespitinizi esas alın.
- Değerlendirin.** Agent artık şunu söyleyebilecek durumda: bu görüntüde üsse 3 km mesafede, 9 m/s hızla yaklaşan bir kamyon var; hakkındaki rapor tespitle uyumlu. Bunun dikkat gerektirip gerektirmediğine, hangi gerekçeyle dikkat gerektirdiğine agent'iniz karar verir.

Verileri kullanırken

- **Veri görüntüye bağlı değildir:** Hareket kayıtları ve raporlar tüm bölge için tek havuzdur. Bir görüntüyle ilgili olanları, çekim saati ve köşe koordinatları üzerinden siz süzersiniz.

- **Konum dönüşümü:** Görüntüler kuşbakışı çekilmiş kabul edilir: üst kenar kuzey, sol kenar batıdır; perspektif düzeltmesi yapılmıştır. Bir pikselin koordinatı, köşe koordinatlarından doğrusal orantıyla bulunur:

```
boylam = sol_üst.boylam + (x / genişlik_px) × (sağ_üst.boylam - sol_üst.boylam)
enlem = sol_üst.enlem + (y / yükseklik_px) × (sol_alt.enlem - sol_üst.enlem)
```

`x, y` pikselin görüntüdeki konumu (sol üst köşe 0,0). Bir aracın konumu için kutusunun merkezini kullanın.

- **Hareket kaydı = son 2 saatin izi:** Kayıtlar 5 dakikalık adımlarla gelir; araçlar dönüş yapar, durur, üs çevresinde dolaşır. Hız ve yönü tek bir adımdan değil, kaydın tamamından okuyun.
- **Hareket kaydının son noktası = görüntüdeki konum:** Her hareket kaydı, ait olduğu görüntünün çekim anında biter. Görüntüdeki bir aracın kaydını ararken, çekim saatindeki (`time` = çekim saati) konumlara bakın; aracınız o konumlardan birinde olmalıdır. Kaydın önceki satırları, aracın oraya nasıl geldiğini anlatır.
- **Araç ve hareket kaydı eşleştirmesi:** Hangi kaydın hangi araca ait olduğu verilmez. Tespit hataları birkaç metrelik sapma yaratabilir; birebir eşitlik aramayın, en yakın noktayı makul bir mesafe sınırı içinde eşleştirin.
- **Eksik eşleşmeler:** Park halindeki araçların hareket kaydı olmayabilir. Kaydı bulunan bir araç da çekim anında görüntü dışında kalmış olabilir.
- **Raporların doğruluğu:** Bazı raporlar doğru, bazıları hatalı veya ilgisizdir. Bunlar işaretlenmez. Raporları kendi tespitlerinizle karşılaştırın.

GLM API Kullanım Rehberi

Yarışma boyunca her takım, OpenAI uyumlu bir gateway üzerinden **Z.ai GLM-5.3-Flash** modelini kullanabilir. Uygulamanızda ya da agent'inizde OpenAI SDK'sını (veya OpenAI uyumlu herhangi bir kütüphaneyi) kullanıyorsanız sadece **base URL**, **API key** ve **model adını** değiştirmeniz yeterli.

BAĞLANTI BİLGİLERİ

Base URL	<code>https://berriailitellm-databasev1826rc3-production-d691.up.railway.app/v1</code>
Model	<code>glm-5.3-flash</code>
API key	Takımınıza özel olarak ayrıca iletilecek (<code>sk-...</code>)

LIMITLER (TAKIM BAŞINA)

Limit	Değer
Toplam bütçe	15 USD (yarışma boyunca, sıfırlanmaz)
İstek / dakika	60
Token / dakika	500.000
Aynı anda istek	4
Context penceresi	~1M token girdi, 128K token çıktı

Bütçe bol, ama sonsuz döngüye giren bir agent'ı fark etmezseniz tükenebilir.

Kalan bütçenizi kontrol edin:

```
curl -s "https://berriailitellm-databasev1826rc3-production-d691.up.railway.app/key/info" \
  -H "Authorization: Bearer sk-..."
# → "spend" (harcanan) ve "max_budget" (15.0) alanlarına bakın
```

HIZLI TEST

```
curl "https://berriailitellm-databasev1826rc3-production-  
d691.up.railway.app/v1/chat/completions" \  
-H "Authorization: Bearer sk-..." \  
-H "Content-Type: application/json" \  
-d '{"model": "glm-5.3-flash", "messages": [{"role": "user", "content":  
"Merhaba!"}]}'
```

PYTHON (OPENAI SDK)

```
pip install openai
```

Basit istek

```
from openai import OpenAI  
  
client = OpenAI(  
    base_url="https://berriailitellm-databasev1826rc3-production-  
d691.up.railway.app/v1",  
    api_key="sk-...",  
)  
  
resp = client.chat.completions.create(  
    model="glm-5.3-flash",  
    messages=[  
        {"role": "system", "content": "Kısa ve net cevap ver."},  
        {"role": "user", "content": "Python'da liste ile tuple farkı  
nedir?"},  
    ],  
    reasoning_effort="low",  
)  
print(resp.choices[0].message.content)
```

Streaming

```
stream = client.chat.completions.create(
    model="glm-5.3-flash",
    messages=[{"role": "user", "content": "Bana kısa bir hikaye anlat."}],
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta
    if delta.content: # düşünce parçaları delta.reasoning_content içinde gelir
        print(delta.content, end="", flush=True)
```

Tool calling (agent döngüsü)

```
import json

def get_weather(city: str) -> str:
    return f"{city}: 22°C, güneşli" # gerçek API çağrınızı buraya koyun

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Bir şehrin güncel hava durumunu döner",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
        },
    },
}]

messages = [{"role": "user", "content": "İstanbul'da hava nasıl?"}]

for _ in range(10): # sonsuz döngüye karşı üst sınır: bütçenizi korur
    resp = client.chat.completions.create(model="glm-5.3-flash",
    messages=messages, tools=tools)
    msg = resp.choices[0].message
    messages.append(msg)
    if not msg.tool_calls:
        print(msg.content)
        break
    for call in msg.tool_calls:
        args = json.loads(call.function.arguments)
        result = get_weather(**args)
        messages.append({"role": "tool", "tool_call_id": call.id,
        "content": result})
```

Görüntü gönderme

Model görüntü okuyabilir. Görüntü, base64'e çevrilip mesajın içinde `image_url` bloğu olarak gönderilir:

```
import base64

with open("foto.jpg", "rb") as f:
    b64 = base64.b64encode(f.read()).decode()

resp = client.chat.completions.create(
    model="glm-5.3-flash",
    messages=[{
        "role": "user",
        "content": [
            {"type": "text", "text": "Bu fotoğrafta neler var?"},
            {
                "type": "image_url",
                "image_url": {"url": f"data:image/jpeg;base64,{b64}"},
            },
        ],
    }],
    reasoning_effort="low",
)
print(resp.choices[0].message.content)
```

- PNG için `data:image/png;base64,...` yazın.
- Token sayısı görüntü boyutuyla artar: 960×540 bir görüntü ~700, 1920×1080 bir görüntü ~2.700 girdi token'ı tutar. `reasoning_effort="low"` ile cevap ~10-15 saniyede gelir.
- Aynı mesaja birden fazla `image_url` bloğu ekleyebilirsiniz.

OPENAI AGENTS SDK (PYTHON)

```
pip install openai-agents
```

```

from openai import AsyncOpenAI
from agents import Agent, Runner, set_default_openai_api,
set_default_openai_client, set_tracing_disabled

set_default_openai_client(AsyncOpenAI(
    base_url="https://berriailitellm-databasev1826rc3-production-
d691.up.railway.app/v1",
    api_key="sk-...",
))
set_default_openai_api("chat_completions") # gateway Responses API
değil, Chat Completions konusu
set_tracing_disabled(True) # tracing OpenAI'nin
sunucularına gider, kapatın

agent = Agent(name="Asistan", instructions="Kısa cevap ver.", model="glm-
5.3-flash")
print(Runner.run_sync(agent, "Merhaba!").final_output)

```

Dosyanıza `agents.py` adını vermeyin. Bu ad kütüphanenin adıyla çakışır ve `ImportError` alırsınız.

ANTHROPIC FORMATI (ANTHROPIC SDK, CLAUDE AGENT SDK VB.)

Gateway Anthropic'in `/v1/messages` formatını da destekler. Anthropic SDK'sında base URL'yi `/v1` **olmadan** verin:

```

from anthropic import Anthropic

client = Anthropic(
    base_url="https://berriailitellm-databasev1826rc3-production-
d691.up.railway.app",
    api_key="sk-...",
)
msg = client.messages.create(
    model="glm-5.3-flash",
    max_tokens=2000,
    messages=[{"role": "user", "content": "Merhaba!"}],
)
print([b.text for b in msg.content if b.type == "text"])

```

HATALAR VE NE YAPMALI

Kod	Anlamı	Ne yapmalı
429	Dakikalık limit ya da aynı anda 4 istek sınırı aşıldı, veya sağlayıcı yoğun	Birkaç saniye bekleyip tekrar deneyin (exponential backoff). Paralel istek sayınızı 4 ile sınırlayın.
400 "Budget has been exceeded"	15 USD bütçe bitti	Organizatörlere yazın.
401	Key geçersiz ya da süresi dolmuş	Key'i doğru kopyaladığınızdan emin olun.
400 "key not allowed to access model"	Model adı yanlış	Model adı tam olarak <code>glm-5.3-flash</code> olmalı.
<code>content</code> boş, <code>finish_reason: "length"</code>	<code>max_tokens</code> düşünmeye yetmedi	<code>max_tokens</code> 'ı artırın ya da kaldırın.

OpenAI SDK, 429 ve 5xx hatalarında zaten otomatik olarak birkaç kez tekrar dener (`max_retries`, varsayılan 2). İsterseniz artırabilirsiniz: `OpenAI(max_retries=5)`.

DESTEKLENMEYENLER

- Embeddings (`/v1/embeddings`)
- Görsel üretme ve ses (image, audio). Görüntü **okuma** desteklenir, yukarıya bakın.
- `glm-5.3-flash` dışındaki modeller

ÖNEMLİ DETAYLAR

GLM-5.3-Flash cevap vermeden önce **her zaman** düşünür (reasoning). Bu yüzden:

1. **Düşünce metni ayrı gelir:** `message.reasoning_content` alanında. Asıl cevap `message.content` içindedir.
2. **max_tokens düşünmeyi de kapsar.** Çok düşük vererseniz (ör. 20) model düşünürken token biter ve `content` **boş** döner (`finish_reason: "length"`). `max_tokens`'ı ya hiç vermeyin ya da cömert tutun (1000+).

3. **Düşünme miktarını** `reasoning_effort` ile ayarlayın: `"low"`, `"high"` veya `"max"`. Basit işlerde `"low"` çok daha hızlı ve ucuzdur. Düşünme tamamen kapatılamaz.
4. `thinking` parametresini **göndermeyin**, hata alırsınız. Bunun yerine `reasoning_effort` kullanın.
5. Cevaplar birkaç saniye sürebilir. Kullanıcıya dönük uygulamalarda **streaming** kullanın.